

Arquitectura de Computadoras — Proyecto 2

Parte 2

Diseño e implementación de la extensión 'C' (instrucciones comprimidas) de RISC-V en un procesador pipelined

“One of my most productive days was throwing away 1,000 lines of code.”

— Ken Thompson

1 Introducción y objetivos

En sistemas embebidos y dispositivos con recursos limitados, el tamaño del código es una restricción crítica: memorias pequeñas, cachés reducidas y anchos de banda limitados hacen que cada byte cuente. Dada esta necesidad, RISC-V cuenta con la extensión 'C' de instrucciones comprimidas, que define un conjunto de instrucciones de 16 bits que reemplazan a las instrucciones de 32 bits más comunes, reduciendo el tamaño del código típicamente entre un 25 % y un 30 % sin sacrificar expresividad.

El objetivo de este proyecto es implementar la extensión 'C' de RISC-V sobre un procesador pipelined. Para ello, los estudiantes deberán diseñar e integrar un descompresor de instrucciones (instruction decompressor) en la etapa de Instruction Fetch o Instruction Decode, capaz de traducir en tiempo real cada instrucción de 16 bits a su equivalente de 32 bits antes de que avance por el pipeline.

Este enunciado corresponde a la segunda parte del proyecto, en la cual se implementará la extensión 'C' sobre el procesador pipelined base de la parte 1.

2 Sobre la extensión 'C' de RISC-V

La extensión 'C' de RISC-V (RVC) define un conjunto de instrucciones de 16 bits que coexisten con las instrucciones de 32 bits del ISA base. No se trata de un modo de operación alternativo que deba activarse o desactivarse, sino de una ampliación del ISA que permite mezclar libremente instrucciones de ambos anchos dentro del mismo flujo de ejecución.

Cada instrucción comprimida tiene exactamente un equivalente de 32 bits al que puede expandirse, lo que simplifica su implementación: se puede agregar un **descompresor** en la etapa de Instruction Fetch o Instruction Decode que traduzca la instrucción de 16 bits a su forma de 32 bits antes de que avance por el pipeline.

La distinción entre instrucciones comprimidas y no comprimidas se realiza mediante los dos bits menos significativos de la instrucción leída: si `instr[1:0] != 2'b11`, la instrucción es de 16 bits y debe ser descomprimida; de lo contrario, se trata de una instrucción estándar de 32 bits. Esta detección tiene además una consecuencia directa sobre el *program counter*: al decodificar una instrucción comprimida, el PC debe avanzar solo 2 bytes en lugar de los 4 habituales.

RVC logra el ahorro de espacio explotando patrones comunes en el código: inmediatos pequeños, uso frecuente de ciertos registros (x8–x15), o coincidencia entre el registro destino y el primer registro fuente.

3 Requerimientos: Extensión 'C'

Se debe **implementar la extensión 'C'** sobre el procesador pipelined de la parte 1 para soportar las instrucciones especificadas en el section [A](#). Estos cambios deben incluir lo siguiente:

1. Un módulo de descompresión de instrucciones de 16 bits.
2. Manejo del incremento del PC ante instrucciones comprimidas.

3. Permitir fetches alineados a 2 bytes.
4. Permitir saltos a direcciones alineadas a 2 bytes, de ser necesario.
5. Cambios al hazard unit, de ser necesario.

El procesador debería finalmente poder ejecutar instrucciones tanto del ISA base como las instrucciones RVC de la section A de forma intercalada, correcta y eficiente.

Como en la parte 1, se pide también escribir algunos programas en RISC-V para comprobar el correcto funcionamiento del CPU. Cada programa escrito debe tener una versión en el ISA base (RV32I), y una versión que use todas las instrucciones comprimidas posibles (RVC) que el procesador soporte. El informe debe mostrar comparativas entre los tamaños resultantes de los programas y el performance (de haber una diferencia).

Algunas ideas de programas de prueba:

- Quicksort (u otros ordenamientos).
- Algoritmos recursivos en árboles completos (ruta mínima, contar nodos, etc.).
- Multiplicación de matrices.

4 Entregables para la parte 2

1. **Diseño en Verilog:** Un archivo .zip con el código fuente del proyecto, programas de Assembly utilizados y testbenches.
2. **Informe:** Informe de la parte 1 con lo siguiente de añadido:
 - Funcionamiento del módulo de descompresión.
 - Cambios realizados al datapath.
 - Cambios realizados al hazard unit si es que hubo, o explicar por qué no hubo cambios.
 - Programas en Assembly utilizados, tanto en sus versiones RV32I como RVC.
 - Comparativa de tamaño de cada programa y su contraparte con instrucciones comprimidas.

5 Referencias

- **Portal de especificaciones de RISC-V oficial:**
<https://riscv.org/technical/specifications/>
- **Especificación del ISA no privilegiado de RISC-V:**
https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf
- **Especificación de la extensión 'C':**
<https://docs.riscv.org/reference/isa/unpriv/c-st-ext.html>

A Instrucciones RVC a implementar

Nótese que **blt** y **bge** no tienen análogo comprimido en RV32C; y que **c.beqz** y **c.bnez** son versiones restringidas de **beq** y **bne** que comparan únicamente contra el registro **x0**.

Instrucción RVC	Equivalente de 32 bits	Descripción
c.lw rd', imm(rs1')	lw rd, imm(rs1)	Load word
c.sw rs2', imm(rs1')	sw rs2, imm(rs1)	Store word
c.lwsp rd, imm(x2)	lw rd, imm(x2)	Load word from stack pointer
c.swsp rs2, imm(x2)	sw rs2, imm(x2)	Store word to stack pointer
c.addi rd, imm	addi rd, rd, imm	Add immediate
c.slli rd, shamt	slli rd, rd, shamt	Shift left logical immediate
c.srli rd', shamt	srli rd, rd, shamt	Shift right logical immediate
c.srai rd', shamt	srai rd, rd, shamt	Shift right arithmetic immediate
c.andi rd', imm	andi rd, rd, imm	AND immediate
c.add rd, rs2	add rd, rd, rs2	Add
c.sub rd', rs2'	sub rd, rd, rs2	Subtract
c.xor rd', rs2'	xor rd, rd, rs2	XOR
c.or rd', rs2'	or rd, rd, rs2	OR
c.and rd', rs2'	and rd, rd, rs2	AND
c.lui rd, nzimm	lui rd, nzimm	Load upper immediate
c.beqz rs1', offset	beq rs1, x0, offset	Branch if zero
c.bnez rs1', offset	bne rs1, x0, offset	Branch si not zero
c.j offset	jal x0, offset	Jump
c.jal offset	jal x1, offset	Jump and link (RV32C only)
c.jr rs1	jalr x0, rs1, 0	Jump register
c.jalr rs1	jalr x1, rs1, 0	Jump and link register

Cuadro 1: Instrucciones RVC a implementar. La notación rd', rs1', rs2' indica registros restringidos al rango x8–x15.